RAGX

Data Quality & Hallucination Detection for RAG Systems

Complete Technical Project Explanation & Viva Master Guide

2. ABSTRACT

Retrieval-Augmented Generation (RAG) systems combine external document retrieval with Large Language Models (LLMs) to ground generated answers. However, standard RAG pipelines suffer from a core vulnerability: they confuse topic similarity with factual proof.

RAGX is an independent Answer Reliability & Hallucination Evaluator designed to audit RAG answers, evaluate Knowledge Base Data Quality, and structure 5-tuple citations (Claim ID, Claim Text, Document Name, Page Number, Chunk ID). RAGX decomposes answers into discrete claims, checks evidence alignment, invokes a Full-KB Retrieval Oracle fallback, classifies failure modes, and calculates a calibrated 60/20/20 reliability score.

3. INTRODUCTION

3.1 The Problem with Standard RAG

A standard RAG system retrieves document passages based on vector similarity and injects them into an LLM prompt. However, high vector similarity indicates that the text discusses the same general topic, NOT that it proves the specific claim generated by the LLM.

RAGX was created to bridge this trust gap. It acts as an independent auditing layer that evaluates whether every generated proposition is factually grounded by document evidence.

4. PROBLEM STATEMENT

RAGX addresses the core engineering problem: 'How can we automatically determine whether an answer generated from retrieved knowledge is actually supported by the available evidence, while also measuring the quality of the knowledge base?'

Common failures in standard RAG include silent hallucinations, incorrect numbers or dates, topic similarity false-positives, missing page citations, and unmonitored Knowledge Base degradation.

5 & 6. EXISTING SYSTEM & LIMITATIONS

5.1 Conventional RAG Pipeline Flow

User Question -> Vector Retrieval -> Context Injection -> LLM Generation -> Answer.

Limitations: Conventional RAG operates blindly after answer generation. It cannot verify whether the context actually supports the answer, leading to silent hallucination propagation.

7, 8 & 9. PROPOSED SYSTEM — RAGX & OBJECTIVES

7.1 The Three Integrated RAGX Phases

- Phase 1 (RAG Workspace): Handles PDF parsing, 500-char chunking, 384-dim SentenceTransformers embedding generation (all-MiniLM-L6-v2), ChromaDB indexing, and RAG Q&A.;
- Phase 2 (Data Quality Auditor): Audits Knowledge Base health S_KB in [0,100] using a ~2ms NumPy matrix dot product (sim_matrix = norm_embs . norm_embs^T).
- Phase 3 (Answer Evaluator): Decomposes answers into claims, checks factual evidence, structures 5-tuple citations, and computes the 60/20/20 reliability score S_Ans.

Core Objectives: Detect unsupported claims, enforce page traceability, evaluate custom user answers, and isolate multi-tenant user data.

10 & 11. END USERS & PRACTICAL APPLICATIONS

10.1 Target User Personas & Use Cases

- Students & Researchers: Verify study document answers and audit page citations.
- Developers & ML Engineers: Benchmark and test RAG pipeline reliability.

- Enterprise Teams: Audit corporate Knowledge Base document quality and prevent AI hallucinations.
- Administrators: Monitor system usage, active sessions, and document security.

Applications include Education, Finance, Legal analysis, Technical documentation, and AI evaluation.

12 & 13. RAGX ARCHITECTURE & WORKFLOW

12.1 Simple Text Architecture Diagram

PDF Upload ---> Document Parsing ---> Chunking (500 chars) ---> Embedding (384-dim) ---> ChromaDB Vector Store

User Question ---> Top-K Vector Search ---> RAG Answer Generation ---> ClaimExtractor ---> EvidenceMatcher ---> Factual Support Check ---> 5-Tuple Citations ---> 60/20/20 Scoring ---> Reliability Result

18-Step Workflow: PDF upload -> Text extraction -> Chunking -> Vector storage -> Data Quality audit -> User question -> Retrieval -> Answer synthesis -> Claim extraction -> Oracle fallback -> Evidence matching -> Support verification -> Aspect analysis -> Failure classification -> 5-Tuple citations -> 60/20/20 scoring -> Risk classification -> UI report.

14. METHODOLOGY OF RAGX

14.1 Stages A through S Methodology

A. Document Ingestion -> B. Text Extraction -> C. Chunk Creation -> D. Embedding Generation -> E. Vector Storage -> F. Retrieval -> G. Answer Generation -> H. Claim Extraction -> I. Evidence Retrieval -> J. Evidence Matching -> K. Proposition Support -> L. Aspect Analysis -> M. Full-KB Oracle -> N. Failure Classification -> O. Citation Traceability -> P. Data Quality Audit -> Q. Reliability Scoring -> R. Risk Classification -> S. Final Presentation.

15 - 17. PHASES 1, 2 & 3 IMPLEMENTATION

15.1 Phase 1 — Document Processing & RAG Workspace: Managed by app/services/document_parser.py, vector_db.py, and rag_engine.py.

16.1 Phase 2 — Data Quality Auditor: Managed by data_quality.py. Calculates Knowledge Base Reliability S_KB = (0.40 * S_ext) + (0.30 * S_div) + (0.30 * S_cons) and performs ~2ms NumPy matrix dot product chunk audit.

17.1 Phase 3 — Answer Reliability Evaluator: Managed by evaluator.py. Combines ClaimExtractor, EvidenceMatcher, QuestionAspectAnalyzer, FullKBRetrievalOracle, FailureClassifier, and 60/20/20 scoring.

18 - 23. CORE DOCUMENT PROCESSING PIPELINE

18.1 Document Parsing: PDF pages parsed maintaining page-level metadata.

19.1 Text Chunking: Fixed size settings.CHUNK_SIZE = 500 characters, overlap = 50 characters.

20.1 Embedding Generation: SentenceTransformers all-MiniLM-L6-v2 model generates 384-dimensional dense vectors.

21.1 ChromaDB Storage: Persistent HNSW vector index storing vectors, document metadata, page numbers, and owner_id.

22.1 Top-K Retrieval: Fetches K=3 most semantically similar text passages.

23.1 Answer Generation: Injects retrieved context passages into LLM prompt for answer synthesis.

24 - 27. CLAIM EXTRACTION & PROPOSITION VERIFICATION

24.1 Claim Extraction: ClaimExtractor breaks answers into discrete factual claims (CLM-001, CLM-002...).

25.1 Evidence Matching: Aligns claims with retrieved context passages.

26.1 Claim Support Verification: Verifies exact phrase matching, normalized substring alignment, and numeric/date set containment.

27.1 Topic Similarity != Factual Support: Cosine vector similarity measures topic closeness; factual support measures truth. High vector similarity with fake claims yields S_supp = 0%.

28 - 32. EVALUATION MODULES & CITATIONS

28.1 & 29.1 Failure Classification: Categorizes failure modes into WELL_GROUNDED, UNSUPPORTED_CLAIMS, RETRIEVAL_FAILURE, EVIDENCE_INSUFFICIENCY, GENERATION_FAILURE.

30.1 5-Tuple Citation Traceability: Structures citations into (1. Claim ID, 2. Claim Text, 3. Document Name, 4. Page Number, 5. Chunk ID / Evidence Info).

31.1 Question Aspect Analysis: Evaluates multi-part query aspect coverage.

32.1 Full-KB Retrieval Oracle: Searches all user chunks if Top-K retrieval misses evidence, distinguishing retrieval failure from generation failure.

33 - 36. SCORING ARCHITECTURE & RELIABILITY FORMULA

33.1 Knowledge Base Reliability Score (S_KB): Evaluates extraction, diversity, and consistency.

34.1 & 35.1 Answer Reliability Score (S_Ans):

*Overall Reliability Score S_Ans = (0.60 * S_supp) + (0.20 * S_cov) + (0.20 * S_sim)*
S_supp = Claim Support (60%), S_cov = Citation Coverage (20%), S_sim = Retrieval Similarity (20%).

36.1 Risk Classification: Score >= 80% -> HIGHLY_RELIABLE (Low Risk); Score 50-79% -> PARTIALLY_RELIABLE; Score < 50% -> UNRELIABLE (Hallucination Risk).

37 - 41. EVALUATION MODES & MULTI-DOMAIN CAPABILITY

37.1 & 38.1 Custom & RAG Answer Modes: Supports benchmarking custom user answers as well as automated RAG outputs.

39.1 Grounded Example: French Revolution in 1789 -> S_supp = 100%, S_Ans = 98% (WELL_GROUNDED).

40.1 Unsupported Example: UN European independence claim -> S_supp = 0%, S_Ans = 19% (UNRELIABLE).

41.1 Multi-Domain Capability: Operates domain-agnostically across History, Science, Math, and Law with ZERO hardcoded rules.

42 - 45. SECURITY, ISOLATION & USER/ADMIN ARCHITECTURE

42.1 & 45.1 Multi-Tenant Request Isolation: Every document, chunk, and log entry is scoped strictly by owner_id across SQLite database queries and ChromaDB vector searches.

43.1 Document Lifecycle: Managed by document_registry.json (active, trash, restore, permanent delete).

44.1 User & Admin Architecture: Authenticated admin accounts (teamragx@gmail.com / teamrag123) inspect user volume, active sessions, and real activity_logs feeds.

46 & 47. TECHNOLOGIES USED & IMPORTANT PROJECT FILES

Technology / File	Role & Implementation Purpose in RAGX
Python 3.11+ / FastAPI	Backend ASGI web framework for REST API endpoints
React 18 & Vite 8.2	Frontend SPA user interface and state management
ChromaDB 0.4+	Persistent HNSW vector database for document embeddings
SentenceTransformers	all-MiniLM-L6-v2 model for 384-dimensional dense vectors
SQLite 3 (WAL Mode)	Relational DB storing users, sessions, and activity logs
app/services/evaluator.py	Phase 3 Answer Reliability & Hallucination Evaluation Engine
app/services/data_quality.py	Phase 2 Knowledge Base Auditor & ~2ms NumPy matrix audit

48 & 49. CODE SNIPPETS & STEP-BY-STEP WORKING EXAMPLE

Snippet 1: Claim Support Verification (app/services/evaluator.py)

```
CODE: def verify_claim_support(claim_text: str, evidence_chunks: list[dict]) -> tuple[str, dict | None, str]: claim_lower = claim_text.lower() for chunk in evidence_chunks: chunk_text = chunk['text'].lower() if claim_lower in chunk_text: return 'SUPPORTED', chunk, 'Exact phrase match found in source chunk.' claim_nums = set(re.findall(r'\b\d+(?:\.\d+)?%?\b', claim_lower)) if claim_nums and claim_nums.issubset(set(re.findall(r'\b\d+(?:\.\d+)?%?\b', chunk_text))): return 'SUPPORTED', chunk, f'Numeric entities {claim_nums} verified in source.' return 'UNSUPPORTED', None, 'No matching proposition evidence found in retrieved chunks.'
```

Explanation: Verifies phrase alignment and numeric entity set containment against retrieved context chunks. Returns SUPPORTED or UNSUPPORTED.

Snippet 2: NumPy Fast Vector Audit (app/services/data_quality.py)

```
CODE: import numpy as np embs_arr = np.array(embeddings) norms = np.linalg.norm(embs_arr, axis=1, keepdims=True) + 1e-9 norm_embs = embs_arr / norms sim_matrix = np.dot(norm_embs, norm_embs.T) # Calculates all pairwise cosine similarities in ~2ms
```

Explanation: Uses matrix multiplication to calculate cosine similarity between all document chunks in RAM in just ~2 milliseconds to detect duplicate text.

50 - 54. TESTING, ADVANTAGES & LIMITATIONS

50.1 Testing & Validation Results

Validated across 4 automated test suites: Auth & Isolation (8/8 PASS), Generic Intent (6/6 PASS), Generic Claim Support (14/14 PASS), Interleaved Queries (12/12 PASS — Zero Leakage).

52.1 System Advantages: Deterministic evaluation, 5-tuple citations, ~2ms vector audit, domain-agnostic engine.

53.1 Limitations: Relies on quality of extracted PDF text; complex multi-step implicit reasoning.

54.1 Future Enhancements: Entailment models, cross-document contradiction reasoning.

55 - 63. COMPREHENSIVE VIVA & EXAMINER QUESTION BANK

55.1 Categories A - X Question Bank (40 Key Questions)

- Q1: What is RAGX? Answer: An Answer Reliability & Hallucination Evaluator for RAG systems that checks if answers are factually supported by documents.
- Q2: Why did you build RAGX? Answer: Standard RAG tools confuse vector similarity with truth. RAGX prevents silent hallucinations by verifying factual claim proof.
- Q3: What is RAG? Answer: Retrieval-Augmented Generation — retrieving document passages to help an LLM answer questions.
- Q4: What is an embedding? Answer: A 384-dimensional dense vector representing text meaning generated by SentenceTransformers.
- Q5: What is ChromaDB? Answer: An open-source vector database using HNSW indexing to store and search text embeddings.
- Q6: What is a 5-tuple citation? Answer: A citation link containing (Claim ID, Claim Text, Document Name, Page Number, Chunk ID).
- Q7: What is the Full-KB Oracle? Answer: A search fallback across all user document chunks if Top-K retrieval misses evidence.
- Q8: Why 60/20/20 scoring? Answer: 60% Claim Support (S_supp), 20% Citation Coverage (S_cov), 20% Retrieval Similarity (S_sim).
- Q9: What is S_KB? Answer: Knowledge Base health score based on extraction health (S_ext), diversity (S_div), and consistency (S_cons).
- Q10: How fast is the vector audit? Answer: ~2 milliseconds using a NumPy matrix dot product in RAM.
- Q11: How is user data isolated? Answer: Filtered strictly by owner_id across SQLite database queries and ChromaDB vector searches.
- Q12: Is RAGX domain-specific? Answer: No, RAGX works domain-agnostically across History, Science, Math, and Law without hardcoded rules.

64 & 65. SYSTEM COMPARISON & EVALUATION ROLE

Dimension	LLM	Standard RAG	RAG Agent	RAGX Engine
What it is	Standalone Model	Retrieval + Generator	Autonomous Loop	Independent Evaluation Wrapper
Main Goal	Text Generation	Context QA	Multi-Step Action	Answer Reliability & Audit
Verification	None	None (Blind Output)	Self-Prompting	Proposition Factual Support Engine
Citations	None	Basic Title	Tool Logs	5-Tuple (Claim, Doc, Page, Chunk)

H2: RAG vs RAGX Distinction

RAG's Job: Retrieve document passages and generate an answer.
RAGX's Job: Evaluate whether that generated answer is actually supported by document evidence.

66. FINAL CONCLUSION

RAGX provides a robust, deterministic, domain-agnostic evaluation wrapper around Retrieval-Augmented Generation systems. By separating semantic vector similarity from proposition-level factual support, providing 100% 5-tuple page-level citation traceability, and auditing Knowledge Base health in ~2ms, RAGX delivers an end-to-end framework for trustworthy AI deployment.